

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ
им. А.И. ГЕРЦЕНА»

Направление подготовки

09.03.01 – Информатика и вычислительная техника

Профиль «Технологии разработки программного обеспечения»

Лабораторная работа №3

«Решение СЛУ методом треугольной факторизации»

Работу выполнили студенты 2 курса 1.1 группы:

Шульга Евгений

Зубов Михаил

Цель: написать программу на языке python, которая реализует решение СЛАУ методом LU – разложения

Оборудование и материалы:

- Python 3.12
- Numpy

Ход работы:

Определение метода:

1. Факторизация матрицы A на L и R :

- Инициализация матриц L и R .
- Задание диагональных элементов матрицы R равными 1.
- Вычисление элементов матриц L и R по рекуррентным формулам, так что $A=LR$.

2. Решение системы линейных уравнений с использованием L и R :

- Прямая подстановка: решение системы $Lz=b$ для z .
- Обратная подстановка: решение системы $Rx=z$ для x .

Код программы:

```
import numpy as np

def triangular_factorization(A):
    n = len(A)
    L = np.zeros((n, n))
    R = np.zeros((n, n))

    # Шаг 1: Инициализация первых элементов
    R[0, 0] = 1
    L[0, 0] = A[0, 0]

    # Шаг 2: Вычисление первого столбца матрицы L и первой строки матрицы R
    for i in range(1, n):
        L[i, 0] = A[i, 0]
        R[0, i] = A[0, i] / L[0, 0]

    # Шаг 3: Вычисление остальных элементов матриц L и R
    for i in range(1, n):
        for j in range(1, i + 1):
            sum_LR = sum(L[i, k] * R[k, j] for k in range(j))
            L[i, j] = A[i, j] - sum_LR

            sum_LR = sum(L[j, k] * R[k, i] for k in range(j))
            R[j, i] = (A[j, i] - sum_LR) / L[j, j]

    return L, R
```

```

def solve_triangular_system(L, R, b):
    n = len(b)
    # Решение  $Ly = b$ 
    y = np.zeros(n)
    for i in range(n):
        y[i] = (b[i] - sum(L[i, k] * y[k] for k in range(i))) / L[i, i]
    print(f"y = {y}\n")
    # Решение  $Rx = y$ 
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - sum(R[i, k] * x[k] for k in range(i + 1, n))) / R[i,
i]

    return x

# Пример системы уравнений
A = np.array([[5, 7, 6, 5],
              [7, 10, 8, 7],
              [6, 8, 10, 9],
              [5, 7, 9, 10]])

b = np.array([23, 32, 33, 31])

# Треугольная факторизация
L, R = triangular_factorization(A)

# Решение системы
solution = solve_triangular_system(L, R, b)
print(f"l =\n {L}\n")
print(f"r =\n {R}\n")
print(f"Ly = b\n {L} * y = {b}")
print(f"Rx = y\n {R} * x = y\n")
print("Решение системы уравнений:", solution)

```

Вывод программы:

```

y = [ 4.6 -1.   2.5  1. ]

l =
[[ 5.   0.   0.   0. ]
 [ 7.   0.2  0.   0. ]
 [ 6.  -0.4  2.   0. ]
 [ 5.   0.   3.   0.5]]

r =
[[ 1.   1.4  1.2  1. ]
 [ 0.   1.  -2.   0. ]
 [ 0.   0.   1.  1.5]
 [ 0.   0.   0.   1. ]]

Ly = b
[[ 5.   0.   0.   0. ]
 [ 7.   0.2  0.   0. ]
 [ 6.  -0.4  2.   0. ]
 [ 5.   0.   3.   0.5]] * y = [23 32 33 31]

Rx = y
[[ 1.   1.4  1.2  1. ]
 [ 0.   1.  -2.   0. ]
 [ 0.   0.   1.  1.5]
 [ 0.   0.   0.   1. ]] * x = y

Решение системы уравнений: [1. 1. 1. 1.]

```

Программой выводится результат факторизации матрицы A, затем, вектор значений X, так же для визуального «удобства» в консоль выводятся формулы использованные для решения СЛУ

Вывод: Мы успешно реализовали алгоритм решения СЛУ методом треугольной факторизации на языке Python